Campus Accessibility Issue Reporter -- Scripts & Steps

All setup, testing, deployment, and demonstration steps

Prepared for BACSE344 Digital Assignment

1. Database Setup Scripts
   schema.sql
   seed.sql
2. Automated API Test Script
3. Testing Table
   How to run these
4. Cloud Deployment Steps
   0. Push your code to GitHub
   1. Database -- confirm it's ready (Neon)
   2. Deploy the backend (Render)
   3. Deploy the frontend (Vercel)
   4. Connect them -- update backend's CORS setting
   5. Final verification checklist
   6. Common deployment errors & fixes
   What to include in your report
5. Demonstration Script
   Before you walk in (do this the night before, not 5 minutes before)
   Script
   If something goes wrong live
   Anticipated questions and short answers

1. Database Setup Scripts

schema.sql

```sql
-- ============================================================
-- Campus Accessibility Issue Reporter -- Database Schema
-- Target: PostgreSQL (Neon / Render Postgres / any Postgres 13+)
-- ============================================================

-- Drop tables if re-running during development (safe order: children first)
DROP TABLE IF EXISTS issue_updates;
DROP TABLE IF EXISTS accessibility_issues;
DROP TABLE IF EXISTS users;

-- ============================================================
-- 1. USERS
-- ============================================================
CREATE TABLE users (
    id              SERIAL PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    email           VARCHAR(150) NOT NULL UNIQUE,
    password_hash   VARCHAR(255) NOT NULL,
    role            VARCHAR(20)  NOT NULL DEFAULT 'student'
                        CHECK (role IN ('student', 'admin')),
    created_at      TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```sql
        -- ============================================================
        -- 2. ACCESSIBILITY ISSUES
        -- ============================================================
        CREATE TABLE accessibility_issues (
            id                      SERIAL PRIMARY KEY,
            user_id                 INTEGER NOT NULL
                                        REFERENCES users(id) ON DELETE
CASCADE,
            title                   VARCHAR(150) NOT NULL,
            description             TEXT NOT NULL,
            category                VARCHAR(30) NOT NULL
                                        CHECK (category IN (
                                            'Wheelchair Ramp',
'Elevator/Lift', 'Accessible Parking',
                                            'Pathway', 'Handrail',
'Entrance', 'Signage',
                                            'Restroom', 'Other'
                                        )),
            location_building       VARCHAR(100),
            location_area           VARCHAR(100),
            location_description    VARCHAR(255),
            latitude                DECIMAL(9,6),
            longitude               DECIMAL(9,6),
            priority                VARCHAR(10) NOT NULL DEFAULT 'Medium'
                                        CHECK (priority IN ('Low', 'Medium',
'High', 'Critical')),
            status                  VARCHAR(20) NOT NULL DEFAULT 'Reported'
                                        CHECK (status IN (
                                            'Reported', 'Under Review',
'Assigned',
                                            'In Progress', 'Resolved',
'Closed'
                                        )),
            assigned_to             VARCHAR(100),
            image_url               VARCHAR(500),
            created_at              TIMESTAMP NOT NULL DEFAULT NOW(),
            updated_at              TIMESTAMP NOT NULL DEFAULT NOW()
        );

        -- ============================================================
        -- 3. ISSUE UPDATES (status/resolution history -- audit trail)
        -- ============================================================
        CREATE TABLE issue_updates (
            id                  SERIAL PRIMARY KEY,
            issue_id            INTEGER NOT NULL
                                    REFERENCES accessibility_issues(id) ON
DELETE CASCADE,
            updated_by          INTEGER NOT NULL
                                    REFERENCES users(id),
            previous_status     VARCHAR(20),
            new_status          VARCHAR(20) NOT NULL,
            comment             TEXT,
            created_at          TIMESTAMP NOT NULL DEFAULT NOW()
        );

        -- ============================================================
        -- 4. Indexes (performance -- evaluators may ask about this)
        -- ============================================================
        CREATE INDEX idx_issues_user_id ON accessibility_issues(user_id);
        CREATE INDEX idx_issues_status ON accessibility_issues(status);
        CREATE INDEX idx_issues_priority ON accessibility_issues(priority);
        CREATE INDEX idx_issues_category ON accessibility_issues(category);
        CREATE INDEX idx_updates_issue_id ON issue_updates(issue_id);

        -- ============================================================
        -- 5. Trigger to auto-update `updated_at` on issue edits
```

```sql
-- ============================================================
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_issues_updated_at
BEFORE UPDATE ON accessibility_issues
FOR EACH ROW
EXECUTE FUNCTION set_updated_at();
```

seed.sql

```sql
-- ============================================================
-- Campus Accessibility Issue Reporter -- Sample / Seed Data
-- Run AFTER schema.sql
-- NOTE: password_hash values below are bcrypt hashes of
"Password123"
--       (generated with bcrypt, 10 rounds) -- safe to use for demo
only.
-- ============================================================

-- ------------------------------------------------------------
-- Users (1 admin, 2 students)
-- ------------------------------------------------------------
INSERT INTO users (name, email, password_hash, role) VALUES
('Admin User',    'admin@campus.edu',
'$2b$10$examplehashforadmin000000000000000000000000000', 'admin'),
('Riya Sharma',   'riya@campus.edu',
'$2b$10$examplehashforriya000000000000000000000000000', 'student'),
('Arjun Mehta',   'arjun@campus.edu',
'$2b$10$examplehashforarjun000000000000000000000000000', 'student');

-- ------------------------------------------------------------
-- Accessibility Issues
-- ------------------------------------------------------------
INSERT INTO accessibility_issues
    (user_id, title, description, category, location_building,
location_area,
     location_description, latitude, longitude, priority, status,
assigned_to, image_url)
VALUES
(2, 'Broken ramp near Block A entrance',
    'The wheelchair ramp at the main entrance of Block A has a large
crack and is unsafe to use.',
    'Wheelchair Ramp', 'Block A', 'Main Entrance', 'Left side of the
main door', 12.925100, 80.183000,
    'High', 'Under Review', NULL, NULL),

(2, 'Elevator not working in Library',
    'The library elevator has been out of service for a week,
blocking access to the 2nd floor reading room.',
    'Elevator/Lift', 'Central Library', '2nd Floor', 'Near the
reference section', 12.926500, 80.184200,
    'Critical', 'Assigned', 'Maintenance Team B', NULL),

(3, 'Missing accessible parking signage',
    'The reserved accessible parking spot near the admin block has
no visible sign, so it is often occupied by regular vehicles.',
    'Signage', 'Admin Block', 'Parking Lot 1', 'Row nearest the
entrance', 12.924800, 80.182500,
    'Medium', 'Reported', NULL, NULL),
```

```
        (3, 'Damaged handrail on Block C stairs',
            'One of the handrail sections near the Block C stairwell is
loose and wobbles when held.',
            'Handrail', 'Block C', 'Stairwell 2', 'Between ground and first
floor', NULL, NULL,
            'Medium', 'Resolved', 'Maintenance Team A', NULL);

        -- ------------------------------------------------------------
        -- Issue Updates (status history / audit trail)
        -- ------------------------------------------------------------
        INSERT INTO issue_updates (issue_id, updated_by, previous_status,
new_status, comment) VALUES
        (1, 1, 'Reported', 'Under Review', 'Site inspection scheduled for
this week.'),
        (2, 1, 'Reported', 'Under Review', 'Confirmed with library staff.'),
        (2, 1, 'Under Review', 'Assigned', 'Assigned to Maintenance Team B
for part replacement.'),
        (4, 1, 'Reported', 'Under Review', 'Checked, handrail is indeed
loose.'),
        (4, 1, 'Under Review', 'In Progress', 'Maintenance Team A
dispatched.'),
        (4, 1, 'In Progress', 'Resolved', 'Handrail bolted and secured.
Verified on site.');
```

ewpage

2. Automated API Test Script

```
        #!/usr/bin/env bash
        # ============================================================
        # Campus Accessibility Issue Reporter -- API Test Script
        # ============================================================
        # Exercises rows 1-21 of docs/testing-table.md against a running
backend.
        #
        # USAGE:
        #   1. Start the backend (npm run dev in backend/) and make sure the
        #      cloud database is reachable.
        #   2. You need ONE existing admin account already in the database -
-
        #      the easiest way is to register a normal student via the app,
        #      then in Neon's SQL Editor run:
        #        UPDATE users SET role = 'admin' WHERE email = 'your-
admin@campus.edu';
        #   3. Run:
        #        API_BASE=http://localhost:5000/api \
        #        ADMIN_EMAIL=your-admin@campus.edu \
        #        ADMIN_PASSWORD=YourPassword123 \
        #        bash api-test-script.sh
        #
        # Every other account this script needs (regular students) is
created
        # fresh on each run with a randomized email, so it's safe to re-run
        # repeatedly without manual cleanup.
        # ============================================================

        API_BASE="${API_BASE:-http://localhost:5000/api}"
        ADMIN_EMAIL="${ADMIN_EMAIL:?Set ADMIN_EMAIL to an existing admin
account email}"
        ADMIN_PASSWORD="${ADMIN_PASSWORD:?Set ADMIN_PASSWORD for that admin
account}"

        PASS=0
```

```
        FAIL=0
        RAND=$RANDOM

        check() {
          local description="$1" expected="$2" actual="$3"
          if [ "$expected" == "$actual" ]; then
            echo "  PASS -- $description (expected $expected, got $actual)"
            PASS=$((PASS+1))
          else
            echo "  FAIL -- $description (expected $expected, got $actual)"
            FAIL=$((FAIL+1))
          fi
        }

        req() {
          # req METHOD PATH [TOKEN] [JSON_BODY]  -> prints http_code, saves
body to /tmp/resp.json
          local method="$1" path="$2" token="$3" body="$4"
          local args=(-s -o /tmp/resp.json -w "%{http_code}" -X "$method"
"${API_BASE}${path}" -H "Content-Type: application/json")
          [ -n "$token" ] && args+=(-H "Authorization: Bearer $token")
          [ -n "$body" ] && args+=(-d "$body")
          curl "${args[@]}"
        }

        echo "=== 1. Database connectivity ==="
        code=$(req GET /health)
        check "GET /health returns 200" "200" "$code"

        echo "=== 2. Registration ==="
        STUDENT_EMAIL="student.${RAND}@campus.edu"
        STUDENT2_EMAIL="student2.${RAND}@campus.edu"
        code=$(req POST /auth/register "" "{\"name\":\"Test
Student\",\"email\":\"${STUDENT_EMAIL}\",\"password\":\"Password123\"}")

        check "Valid registration returns 201" "201" "$code"
        STUDENT_TOKEN=$(grep -o '"token":"[^"]*' /tmp/resp.json | cut -d'"'
-f4)

        code=$(req POST /auth/register "" "{\"name\":\"Test
Student\",\"email\":\"${STUDENT_EMAIL}\",\"password\":\"Password123\"}")

        check "Duplicate email registration returns 409" "409" "$code"

        code=$(req POST /auth/register "" "{\"name\":\"No
Password\",\"email\":\"nopass.${RAND}@campus.edu\"}")
        check "Registration with missing password returns 400" "400" "$code"

        code=$(req POST /auth/register "" "{\"name\":\"Test Student
2\",\"email\":\"${STUDENT2_EMAIL}\",\"password\":\"Password123\"}")
        STUDENT2_TOKEN=$(grep -o '"token":"[^"]*' /tmp/resp.json | cut -d'"'
-f4)
        check "Second student registration returns 201" "201" "$code"

        echo "=== 3. Login ==="
        code=$(req POST /auth/login "" "
{\"email\":\"${STUDENT_EMAIL}\",\"password\":\"Password123\"}")
        check "Valid login returns 200" "200" "$code"

        code=$(req POST /auth/login "" "
{\"email\":\"${STUDENT_EMAIL}\",\"password\":\"WrongPassword\"}")
        check "Wrong password returns 401" "401" "$code"

        code=$(req POST /auth/login "" "
{\"email\":\"nobody.${RAND}@campus.edu\",\"password\":\"Password123\"}")
```

```
        check "Unknown email returns 401" "401" "$code"

        echo "=== 4. Create issue ==="
        code=$(req POST /issues "$STUDENT_TOKEN" '{"title":"Test ramp
issue","description":"Automated test","category":"Wheelchair
Ramp"}')
        check "Authenticated create returns 201" "201" "$code"
        ISSUE_ID=$(grep -o '"id":[0-9]*' /tmp/resp.json | head -1 | grep -o
'[0-9]*')

        code=$(req POST /issues "" '{"title":"No auth","description":"Should
fail","category":"Pathway"}')
        check "Unauthenticated create returns 401" "401" "$code"

        code=$(req POST /issues "$STUDENT_TOKEN" '{"title":"Bad
category","description":"Should fail","category":"Not A Category"}')
        check "Invalid category returns 400" "400" "$code"

        echo "=== 5. View issue ==="
        code=$(req GET /issues "$STUDENT_TOKEN")
        check "GET own issues returns 200" "200" "$code"

        code=$(req GET "/issues/${ISSUE_ID}" "$STUDENT_TOKEN")
        check "GET single issue as owner returns 200" "200" "$code"

        code=$(req GET "/issues/${ISSUE_ID}" "$STUDENT2_TOKEN")
        check "GET single issue as non-owner returns 403" "403" "$code"

        echo "=== 6. Update issue ==="
        code=$(req PUT "/issues/${ISSUE_ID}" "$STUDENT_TOKEN"
'{"title":"Updated title"}')
        check "Update own issue while Reported returns 200" "200" "$code"

        code=$(req PUT "/issues/${ISSUE_ID}" "$STUDENT2_TOKEN"
'{"title":"Hijacked"}')
        check "Update someone else's issue returns 403" "403" "$code"

        echo "=== 7. Unauthorized / admin access ==="
        code=$(req GET /issues "")
        check "No token on protected route returns 401" "401" "$code"

        code=$(req GET /admin/issues "$STUDENT_TOKEN")
        check "Student calling admin route returns 403" "403" "$code"

        echo "=== 8. Admin login + access ==="
        code=$(req POST /auth/login "" "
{\"email\":\"${ADMIN_EMAIL}\",\"password\":\"${ADMIN_PASSWORD}\"}")
        ADMIN_TOKEN=$(grep -o '"token":"[^"]*' /tmp/resp.json | cut -d'"' -
f4)
        check "Admin login returns 200" "200" "$code"

        code=$(req GET /admin/issues "$ADMIN_TOKEN")
        check "Admin can list all issues (200)" "200" "$code"

        code=$(req GET /admin/dashboard "$ADMIN_TOKEN")
        check "Admin dashboard returns 200" "200" "$code"

        echo "=== 9. Status / priority / assignment ==="
        code=$(req PUT "/admin/issues/${ISSUE_ID}/status" "$ADMIN_TOKEN"
'{"status":"Under Review","comment":"Automated test comment"}')
        check "Valid status update returns 200" "200" "$code"

        code=$(req PUT "/admin/issues/${ISSUE_ID}/status" "$ADMIN_TOKEN"
'{"status":"Not A Status"}')
        check "Invalid status value returns 400" "400" "$code"
```

```
        code=$(req PUT "/admin/issues/${ISSUE_ID}/priority" "$ADMIN_TOKEN"
'{"priority":"Critical"}')
        check "Valid priority update returns 200" "200" "$code"

        code=$(req PUT "/admin/issues/${ISSUE_ID}/assign" "$ADMIN_TOKEN"
'{"assigned_to":"Maintenance Team A"}')
        check "Assignment returns 200" "200" "$code"

        echo "=== 10. Update history ==="
        code=$(req GET "/issues/${ISSUE_ID}/updates" "$STUDENT_TOKEN")
        check "Owner can view update history (200)" "200" "$code"

        code=$(req GET "/issues/${ISSUE_ID}/updates" "$STUDENT2_TOKEN")
        check "Non-owner blocked from update history (403)" "403" "$code"

        code=$(req POST "/issues/${ISSUE_ID}/updates" "$STUDENT_TOKEN"
'{"new_status":"Resolved"}')
        check "Student cannot POST an update (403)" "403" "$code"

        echo "=== 11. Cleanup / delete ==="
        code=$(req DELETE "/admin/issues/${ISSUE_ID}" "$ADMIN_TOKEN")
        check "Admin delete returns 200" "200" "$code"

        echo "=== 12. API error handling ==="
        code=$(req GET /not-a-real-route "")
        check "Unknown route returns 404" "404" "$code"

        echo ""
        echo "========================================"
        echo "  RESULTS: $PASS passed, $FAIL failed"
        echo "========================================"
        [ "$FAIL" -eq 0 ] && exit 0 || exit 1
```

ewpage

3. Testing Table

Manual + scripted test cases covering every flow required by the assignment brief (Section 23). "Actual result" and "Status" columns are left for you to fill in as you run each test during your own verification pass -- the automated script in `api-test-script.sh` runs most of the API-level cases for you and prints PASS/FAIL directly.

#	Feature	Test Case	Steps / Input
1	Registration	Valid registration	POST `/api/auth/register` with unique name/email/password >=6 chars
2	Registration	Duplicate email	Register twice with the same email
3	Registration	Missing fields	Omit `password` from request body
4	Login	Valid login	POST `/api/auth/login` with correct email/password
5	Login	Invalid login (wrong password)	Correct email, wrong password
6	Login	Invalid login (unknown email)	Email not in the database
		Create as	

7	Create issue	authenticated student	POST `/api/issues` with valid token + required fields
8	Create issue	Unauthenticated create	POST `/api/issues` with no `Authorization` header
9	Create issue	Invalid category	POST with `category: "Not A Real Category"`
10	View issue	View own issues list	GET `/api/issues` as the owner
11	View issue	View single issue (owner)	GET `/api/issues/:id` as the reporting student
12	View issue	View single issue (non-owner, non-admin)	GET `/api/issues/:id` as a different student
13	Update issue	Update own issue while `Reported`	PUT `/api/issues/:id` as owner, status still `Reported`
14	Update issue	Update after status has changed	PUT `/api/issues/:id` once status != `Reported`
15	Update issue	Update someone else's issue	PUT `/api/issues/:id` as a non-owner
16	Delete issue	Delete own issue while `Reported`	DELETE `/api/issues/:id` as owner
17	Delete issue	Delete after status has changed	DELETE once status != `Reported`
18	Unauthorized access	Student calls admin-only route	GET `/api/admin/issues` with a student token
19	Unauthorized access	No token on protected route	GET `/api/issues` with no token
20	Admin access	Admin views all issues	GET `/api/admin/issues` with an admin token
21	Admin access	Admin dashboard stats	GET `/api/admin/dashboard` with an admin token
22	Status update	Admin changes issue status	PUT `/api/admin/issues/:id/status` with valid status + comment
23	Status update	Invalid status value	PUT status with `status: "Not A Status"`
24	Priority update	Admin changes priority	PUT `/api/admin/issues/:id/priority` with a valid value
25	Priority update	Invalid priority value	PUT priority with `priority: "Urgent!"`
26	Assignment	Admin assigns an issue	PUT `/api/admin/issues/:id/assign` with `assigned_to`
27	Image upload	Valid image upload	POST `/api/uploads/image` with a JPEG/PNG under 5MB, authenticated
28	Image upload	Oversized file	Upload a file > 5MB
29	Image upload	Non-image file	Upload a `.pdf` or `.txt`

30	Image upload	Unauthenticated upload	POST `/api/uploads/image` with no token
31	Database connectivity	Health check	GET `/api/health`
32	Database connectivity	Health check with DB unreachable	Stop the DB / use a bad `DATABASE_URL`, then GET `/api/health`
33	API errors	Unknown route	GET `/api/not-a-real-route`
34	API errors	Malformed JSON body	POST with broken JSON syntax
35	Update history	Student views own update history	GET `/api/issues/:id/updates` as the reporting student
36	Update history	Non-owner views update history	GET `/api/issues/:id/updates` as a different student
37	Update history	Student attempts to POST an update	POST `/api/issues/:id/updates` with a student token

How to run these

- **Automated (recommended for most rows):** run `api-test-script.sh` against your local or deployed backend -- see instructions in that file's header comment. It exercises rows 1-21 and prints a PASS/FAIL summary.
- **Manual (image upload, DB-down case, UI-driven cases):** use Postman with the provided collection, or the running frontend directly -- rows 27-30, 32, and any UI-level check are easiest to verify by hand.
- Fill in the "Actual Result" and "Status" columns as you go; this table is designed to be pasted directly into your assignment report's Testing section (Section 23) once completed.

ewpage

4. Cloud Deployment Steps

This walks through getting the whole project live at a public URL. You already have: - Database on **Neon** (from Phase 2) - Image storage on **Cloudinary** (from Phase 7)

What's left: push your code to GitHub, deploy the **backend** to Render, deploy the **frontend** to Vercel, then connect the two together. Total time: ~20 minutes.

---

0. Push your code to GitHub

Both Render and Vercel deploy by connecting to a GitHub repository, so this has to happen first.

1. Go to **github.com** -> sign in -> click the **+** icon (top right) -> **New repository**.

2. Name it `campus-accessibility-reporter` -> keep it **Public** or **Private** (either works) -> **Create repository**. Don't check "Add a README" -- you'll push your existing folder.

3. On your own computer, open a terminal in your project's root folder (the one containing `frontend/`, `backend/`, `database/`, `docs/`) and run:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/<your-username>/campus-accessibility-reporter.git
git push -u origin main
```

4. **Before this step**, make sure `.env` files are NOT committed. Create a `.gitignore` in your project root containing:

```
node_modules/
.env
dist/
```

   If you already ran `git add .` before creating `.gitignore`, run `git rm -r --cached backend/.env frontend/.env` (if they exist) before committing.

5. Refresh your GitHub repo page -- you should see `frontend/`, `backend/`, `database/`, `docs/` folders there.

---

1. Database -- confirm it's ready (Neon)

You already set this up in Phase 2. Just double-check: 1. Log into **neon.tech** -> open your project -> confirm the connection string is still visible under **Connection Details**. 2. Keep this tab open -- you'll paste this string into Render in step 2.

---

2. Deploy the backend (Render)

1. Go to **render.com** -> sign up/log in (GitHub login is easiest -- it'll also grant Render access to your repos).

2. Click **New +** -> **Web Service**.

3. Connect your `campus-accessibility-reporter` repository (you may need to click "Configure account" to grant Render access to it first).

4. Fill in the settings: | Field | Value | |---|---| | Name | `campus-accessibility-backend` | | **Root Directory** | `backend` <- important, since backend and frontend share one repo | | Runtime | Node | | Build Command | `npm install` | | Start Command | `npm start` | | Instance Type | Free |

5. Scroll to **Environment Variables** -> click **Add Environment Variable** and add each of these (values from your local `backend/.env`): | Key | Value | |---|---| | `DATABASE_URL` | your Neon connection string | | `JWT_SECRET` | your long random string | | `CLOUDINARY_CLOUD_NAME` | from Cloudinary dashboard | | `CLOUDINARY_API_KEY` | from Cloudinary dashboard | | `CLOUDINARY_API_SECRET` | from Cloudinary dashboard | | `FRONTEND_ORIGIN` | leave as `http://localhost:5173` for now -- you'll

update this in step 4 | *(`PORT` doesn't need to be set -- Render sets its own and your `server.js` already reads `process.env.PORT` with a fallback.)*

6. Click **Create Web Service**. Render will build and deploy -- this takes 2-5 minutes. Watch the logs; you're looking for `Server running on port ....`

7. Once live, Render gives you a public URL like:

   `https://campus-accessibility-backend.onrender.com`

8. **Test it immediately:** open `https://campus-accessibility-backend.onrender.com/api/health` in your browser. You should see:

   ```
   {"success":true,"message":"API is running","database":"connected"}
   ```

   If you see `"database not reachable"`, double check the `DATABASE_URL` env var for typos.

   Free-tier note: Render's free web services "spin down" after 15 minutes of inactivity and take ~30-60 seconds to wake up on the next request -- expected behavior, not a bug. Worth mentioning during your demo if the first request feels slow.

---

3. Deploy the frontend (Vercel)

1. Go to **vercel.com** -> sign up/log in with GitHub.

2. Click **Add New...** -> **Project**.

3. Import your `campus-accessibility-reporter` repository.

4. Configure: | Field | Value | |---|---| | Framework Preset | Vite (should auto-detect) | | **Root Directory** | `frontend` <- click "Edit" next to Root Directory and select it | | Build Command | `npm run build` (default, leave as-is) | | Output Directory | `dist` (default, leave as-is) |

5. Expand **Environment Variables** -> add: | Key | Value | |---|---| | `VITE_API_BASE_URL` | `https://campus-accessibility-backend.onrender.com/api` (your Render URL from step 2, with `/api` on the end) |

6. Click **Deploy**. Takes about a minute.

7. Once done, Vercel gives you a public URL like:

   `https://campus-accessibility-reporter.vercel.app`

---

4. Connect them -- update backend's CORS setting

Right now your backend only accepts requests from `http://localhost:5173`. Update it to also accept your live frontend URL:

1. Go back to your **Render** dashboard -> your backend service -> **Environment**.

2. Edit `FRONTEND_ORIGIN` -> change its value to your Vercel URL:

```
https://campus-accessibility-reporter.vercel.app
```

3. Save -- Render will automatically redeploy with the new value (takes ~1 minute).

---

5. Final verification checklist

Open your live Vercel URL in a browser and walk through:

- ☐ Landing page loads
- ☐ Register a new student account -> redirected to dashboard
- ☐ Refresh the page -> still logged in
- ☐ Report an issue with a photo -> succeeds, photo preview shows "uploaded successfully"
- ☐ Issue appears in "My Reports"
- ☐ Open the issue -> details, status timeline, empty update history all render
- ☐ In Neon's SQL Editor, run `SELECT * FROM accessibility_issues;` -> confirm the row is really there
- ☐ Promote your test account to admin via SQL, log out, log back in
- ☐ Admin Dashboard shows correct stats
- ☐ Change the issue's status + add a comment -> save
- ☐ Log back in as the student -> update history now shows that change

If every box checks, your app is fully live and cloud-deployed end to end.

---

6. Common deployment errors & fixes

Symptom	Likely cause	Fix
Frontend loads but every request fails, browser console shows a CORS error	`FRONTEND_ORIGIN` on Render doesn't match your Vercel URL exactly	Re-check for trailing slashes / `http` vs `https` mismatch
Vercel build fails with "vite: not found"	Root Directory wasn't set to `frontend`	Go to Project Settings -> General -> Root Directory -> set to `frontend`, redeploy
Render deploy fails immediately	Root Directory wasn't set to `backend`, or `package.json`'s `start` script is missing	Confirm Root Directory = `backend`; confirm `"start": "node src/server.js"` exists
`/api/health` shows `database: not reachable`	Wrong or expired `DATABASE_URL`	Copy the connection string fresh from Neon, re-paste into Render env vars
First request after inactivity takes 30-60 seconds	Render free-tier cold start	Expected -- mention this in your demo, or open the health-check URL a minute before presenting
Login works locally but not on the deployed site	`VITE_API_BASE_URL` on Vercel still points to `localhost`	Fix the env var on Vercel, then redeploy (Vercel -> Deployments -> ... -> Redeploy)

Image upload fails only in production	Cloudinary env vars weren't copied over to Render	Re-add all three `CLOUDINARY_*` variables in Render's Environment tab

---

What to include in your report

- Your live URLs: `https://campus-accessibility-reporter.vercel.app` (frontend) and `https://campus-accessibility-backend.onrender.com` (backend API)
- A screenshot of the Render and Vercel dashboards showing successful deploys
- A screenshot of `/api/health` returning `"database":"connected"` in production

ewpage

5. Demonstration Script

Target: 5-7 minutes. Timings below are guides, not hard stops -- adjust pacing to your evaluator's questions.

Before you walk in (do this the night before, not 5 minutes before)

- ☐ Confirm the live URLs both load: frontend (Vercel) and `/api/health` on the backend (Render).
- ☐ **Hit the Render health-check URL ~1 minute before your slot** -- free-tier cold start takes 30-60s; you don't want that delay eating into your live demo.
- ☐ Have one admin account ready (promoted via SQL beforehand -- don't do this live, it looks fiddly).
- ☐ Delete/clean up test junk from earlier testing so the admin dashboard doesn't show obviously fake data (or don't -- real seeded data works fine too).
- ☐ Have Postman open in a second tab/window with the collection imported and a token already pasted in, so you're not typing during the demo.
- ☐ Have Neon's SQL Editor open in a third tab, ready to run `SELECT * FROM accessibility_issues ORDER BY id DESC LIMIT 5;`.
- ☐ Have your architecture diagram (from Phase 1) ready to screen-share or show as a slide.

Script

1. Introduce the problem (30 sec)

> "Accessibility issues on campus -- broken ramps, elevators, missing signage -- usually get reported informally, if at all, with no way to track what happens next. This app gives students and staff a structured way to report these issues and see them through to resolution."

2. Register / log in as a student (30 sec)

- Open the live frontend.
- Register a new account live (use an email like `demo.[yourname]@campus.edu` -- fresh, so it's clearly not seeded data).
- Point out: lands directly on the Student Dashboard, session persists on refresh.

3. Create an accessibility issue (60 sec)

- Click "Report an Issue."
- Fill in a realistic example (e.g., "Broken ramp near Block A entrance," category Wheelchair Ramp, building/area fields).
- **Upload a photo** -- narrate: "This uploads directly to Cloudinary, cloud object storage -- the database only stores the resulting URL, not the image itself."
- Submit -> redirected to My Reports, issue visible.

4. Show it in the dashboard (20 sec)

- Back to Student Dashboard -- point out the stat counts updating, the recent-reports list.

5. Show the database record (30 sec)

- Switch to the Neon tab, run the `SELECT` query.
- "Same row, same data -- the frontend and the database are talking to the same live cloud instance, from wherever you are."

6. Log in as administrator (20 sec)

- Log out, log in with the pre-promoted admin account.
- Land on Admin Dashboard -- point out it's a different view entirely, enforced by role-based access control in the JWT.

7. Show the admin dashboard (30 sec)

- Point out the stat cards (Total, Reported, Under Review, In Progress, Resolved, Critical).
- Open "All Issues" -- demonstrate a filter (e.g., filter by category or search by title).

8. Change priority/status, add a resolution note (60 sec)

- Open the issue you just created (Admin Issue Details).
- Change status to "Under Review," add a comment like "Site inspection scheduled."
- Change priority to "High."
- Save -- point out this is a single database transaction: the issue *and* the audit-trail row are written together.

9. Return to student account, show updated status (30 sec)

- Log out, log back in as the student.
- Open the same report -- status badge updated, update history now shows the admin's comment with a timestamp.
- "This is the transparency piece -- the reporter doesn't have to follow up, they can just check."

10. Demonstrate the REST API directly (45 sec)

- Switch to Postman.
- Run `GET /api/admin/issues` -- show the raw JSON response.
- Run one of the earlier requests again to show it's stateless (any client, not just the React app, can talk to this API).
- Optionally: try the same admin request with a student token to show `403 Forbidden` -- a nice, fast way to demonstrate role-based authorization live.

11. Show the cloud architecture (30 sec)

- Show the architecture diagram slide/screen: Frontend (Vercel) -> Backend API (Render) -> Database (Neon), with Cloudinary alongside.
- "Three separate managed cloud services, each independently deployed and scalable -- that's the cloud-computing piece of this assignment."

12. Close (15 sec)

> "So end to end: a student reports an issue with a photo, it's stored in a cloud database with the image in cloud object storage, an administrator triages and resolves it through a REST API enforcing role-based access, and the student sees the update -- all through a fully cloud-deployed application."

If something goes wrong live

- **Render is slow to respond:** you already pinged it beforehand, but if it still happens -- narrate it. "This is the free tier's cold start -- in a paid deployment this wouldn't happen." Evaluators respect an honest, calm explanation over silence.
- **A field doesn't save as expected:** don't panic-debug live. Say "let me show you that same flow via Postman instead" and pivot to the API directly -- it's more reliable and still demonstrates the same functionality.
- **Wi-Fi drops:** have `npm run dev` running locally on your laptop as a backup, pointed at the same Neon database -- same data, no internet dependency for the demo itself (only needed if evaluators specifically want to see the public URL).

Anticipated questions and short answers

Question	Answer
Why PostgreSQL over MongoDB?	The data is relational -- issues belong to users, updates belong to issues -- foreign keys and constraints give real data integrity that a document store would have to enforce in application code instead.
Why is the image not in the database?	Databases are inefficient at large binary blobs; object storage is purpose-built for files and serves them via CDN. The DB only needs a lightweight URL string.
How does the system scale?	Stateless JWT auth means any number of backend instances can serve any request -- no server-side session to keep in

	sync. Database and storage are managed services that scale independently of the backend.
What stops a student from editing someone else's report?	Every issue route checks `req.user.id` (from the verified JWT) against the issue's `user_id` before allowing an edit or delete -- enforced server-side, not just hidden in the UI.
What happens to the audit trail if two admins update at once?	Status changes are wrapped in a database transaction -- the status update and the `issue_updates` row are written atomically, so there's no window where one succeeds without the other.